



به نام خدا

دانشکده‌ی مهندسی برق و کامپیوتر دانشکده فنی دانشگاه تهران

مبانی کامپیوتر و برنامه نویسی



اساتید:
دکترمرادی، دکتر هاشمی

عنوان:
لیست‌های پیوندی 2

نیمسال اول
1402-03

در آزمایشگاه گذشته، تا حدودی با مفهوم اضافه کردن¹ در لیست‌های پیوندی آشنا شدید. حال به ادامه‌ی بررسی لیست‌های پیوندی می‌پردازیم.

1- انجام دهید! (اضافه کردن)

برنامه‌ای را که به ضمیمه این فایل در اختیارتان قرار گرفته است، در یک پروژه‌ی جدید اجرا کنید. عملکرد هریک از توابع را برای دستیاران آموزشی توضیح دهید.

حال باتوجه به روشی که در آزمایشگاه قبل یادگرفتید، تابع زیر را به نحوی تکمیل کنید که با گرفتن اشاره‌گر به یک گره و اشاره‌گر به ابتدای لیست پیوندی (head) و عدد n ، آن گره را به خانه‌ی $n+1$ ام لیست پیوندی اضافه کند. (تضمین می‌شود مقدار n از تعداد گره‌های اولیه‌ی لیست بیشتر نیست).

```
int InsertNewNode (information *new_node , information *head , int n)
{
    information *temp;
    temp = head;
    // Shifting the pointer
    for(int i =0; i<n; i++){
        temp = ...;
    }
    // Insertion
    ...
    return 1;
}
```

این تابع را به برنامه اضافه کرده و برای بررسی عملکرد آن قطعه کد زیر را به انتهای تابع main اضافه کنید.

```
// Inserting new node
InsertNewNode(CreateNode(33,54) , head , 4);
printInformation (head);
```

¹ Insertion

✚ فرض کنید مقدار n وارد شده بیشتر از تعداد گره‌های اولیه‌ی لیست باشد. پس قطعه کد زیر را به‌طوری تکمیل کرده و به انتهای حلقه‌ی `for` موجود در تابع اضافه کنید که در صورت نامناسب بودن مقدار n ، پیغام خطا نمایش داده شده و مقدار 0 را برگشت دهد.

```
if (temp == ... ){
    printf("Invalid n value! The insertion was unsuccessful!\n");
    ...; // Quitting the function
}
```

عملکرد برنامه را بررسی کنید.

✓ پاسخ تمام بخش‌های بالا را با دستیاران آموزشی درمیان بگذارید.(قسمت 1)

2- انجام دهید!(حذف کردن)²

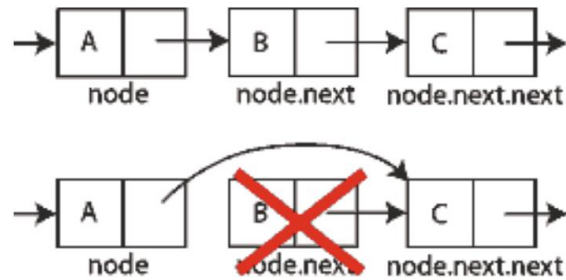
✚ حال می‌خواهیم تابعی را به برنامه اضافه کنیم تا خانه‌ی n ام لیست پیوندی را حذف کند و در صورت موفقیت‌آمیز بودن مقدار 1 و در غیر این‌صورت مقدار 0 را برگرداند. بدین منظور تابع زیر را کامل کنید.

```
int Delete(information *head , int n)
{
    information *current , *previous;
    current = head;
    // Shifting the pointer
    for(int i =0; i<n; i++){
        previous = ...;
        current = ...;
        if (current == ...){
            printf("Invalid n value! The deletion was unsuccessful!\n");
            ...; // Quitting the function
        }
    }
    // Deleting
    ...;
    free(current);
    return 1;
}
```

برای بررسی عملکرد برنامه پس از فراخوانی این تابع، از تابع `PrintInformation` استفاده کنید.

² Deletion

راهنمایی: برای درک بیشتر عملیات حذف کردن، به شکل زیر توجه کنید:



چرا در برنامه‌ی بالا از تابع `free` استفاده کرده‌ایم؟ در مورد عواقب عدم استفاده از آن تحقیق کرده و به دستیاران آموزشی توضیح دهید.

نتیجه را با دستیاران آموزشی درمیان بگذارید. (قسمت 2) ✓

3- انجام دهید! (جستجو کردن³) ↩

اکنون قصد داریم تابعی ایجاد کنیم که با دریافت پارامتر اول یک گره از لیست پیوندی، آن گره را یافته و مقدار پارامتر دوم آن را نمایش داده و مقدار 1 را برگرداند. همچنین در صورتی که گره با پارامتر اول داده شده موجود نبود، پیام خطا نمایش داده و مقدار 0 را برگرداند. این تابع را کامل کنید:

```
int Search(information *head , float parameter_value)
{
    information *temp;
    temp = head->next;
    // Searching
    while((...) && (temp->next != ...))
    {
        temp = ...; // Shifting the pointer
    }
    if (temp->first_parameter == ...){
        printf("the second parameter = %f\n" , ...);
        ...;
    }else{
        printf("your entered parameter, does not exist!\n");
        ...;
    }
}
```

³ Search

✚ حال با توجه به آنچه در این قسمت فراگرفتید، تابع حذف کردن را طوری پیاده سازی کنید که با گرفتن مقدار پارامتر اول، خانه‌ی متناظر با آن را حذف کند. (در واقع این تابع به جای گرفتن شماره‌ی خانه‌ای که باید آن را حذف کند، مقدار پارامتر اول آن را به عنوان ورودی دریافت می‌کند).

✓ پاسخ هر دو بخش بالا را با دستیاران آموزشی درمیان بگذارید. (قسمت 3)

✓ لینک کاربردی

تا کنون با لیست‌های پیوندی یک‌طرفه⁴ کار کردیم که ساختار آن‌ها به شکل زیر بود:



اکنون با انواع دیگری از لیست‌های پیوندی آشنا خواهیم شد:

1- **لیست‌های پیوندی دو طرفه⁵:** نوع خاصی از لیست پیوندی است که در آن هر گره علاوه بر اشاره گر به گره بعدی، دارای اشاره گر به گره قبلی نیز می‌باشد.



2- **لیست‌های پیوندی حلقوی⁶:** یک لیست پیوندی است که در آن همه گره‌ها برای تشکیل یک حلقه به هم متصل می‌شوند. در یک لیست پیوندی حلقوی، اولین گره و آخرین گره به یکدیگر متصل هستند به صورتی که یک دایره را تشکیل می‌دهند. در پایان این لیست پیوندی NULL وجود ندارد.



✓ در مورد کاربردهای انواع لیست‌های پیوندی تحقیق کنید.

⁴ Singly linked list

⁵ Doubly linked list

⁶ Circular linked list

4- انجام دهید!

فرض کنید قصد ساخت یک لیست پیوندی دوطرفه را داریم (یعنی علاوه بر اشاره گر `*next`، یک اشاره گر `*previous` نیز در هر گره وجود دارد که باید به گره پیشین اشاره کند). تابع زیر را که اشاره گر `head` و مقدار `n` را می گیرد، طوری تکمیل کنید که یک لیست پیوندی دوطرفه با `n` گره را ایجاد کند.

```
int CreateNode(information* head , int n)
{
    if (head == ...)
        return 0;
    information *current_node = head , *next_node;
    for(int i=0; i<n; i++)
    {
        // Allocating memory
        next_node = ...;
        // Reordering pointers
        ...
    }
    return 1;
}
```

می خواهیم توسط یک تابع، گره `new_node` را بعد از گره `temp` اضافه کنیم. به نظرتان قطعه کد زیر این کار را به درستی انجام می دهد؟ اگر پاسخ منفی است، آن را اصلاح کنید.

```
// Insertion
temp->next = new_node;
new_node->previous = temp;
new_node->next = temp->next;
temp->next->previous = new_node;
```

راهنمایی: تنها کافیست ترتیب قرارگیری این خطوط را تغییر دهید.

✚ حال فرض کنید قصد داریم گره the_node را از یک لیست پیوندی دوطرفه حذف کنیم. قطعه کد زیر را کامل کنید.

```
// Deleting
... = the_node->next;
... = the_node->previous;
free(...);
```

✔ نتایج را به دستیاران آموزشی نشان دهید.(قسمت 4)

✓ برای مطالعه در مورد عملیات حذف کردن و اضافه کردن در لیست‌های حلقوی، می‌توانید به این [لینک](#) مراجعه کنید.

موفق باشید.

تهیه و تنظیم: امیرمرتضی رضائی